1　データベース設計

2　概要

本レポートでは，MATLAB を用いて行った全 5 回の音響信号処理および画像処理の実験についてまとめる． 実験では，純音やうなり，フーリエ級数などの音響信号の基礎から，画像の量子化，閾値処理，フィルタ処理，さらに 2 次元 FFT を用いた周波数解析までを行った． 各実験で得られた波形や画像を比較し，それぞれの処理によって信号や画像がどのように変化するかを考察した．

3　目的

本実験の目的は，MATLAB を用いて音響信号および画像データに対する基本的な信号処理を行い，それぞれの処理方法とその特徴を理解することである． 第 1 回では，正弦波の生成を通して周波数，振幅，位相の違いを確認し，うなり，フーリエ級数，白色ガウス雑音について学ぶ． 第 2 回では，FFT を用いて信号の周波数成分を解析し，周波数領域でのフィルタ処理について理解する．第 3 回では，RGB 画像の構造，量子化，二値化，ヒストグラム，背景差分などの基本的な画像処理を行う． 第 4 回では，平均化フィルタやメディアンフィルタによる雑音除去，ソーベルフィルタやラプラシアンフィルタによるエッジ検出，HSV 色空間を用いた色領域の抽出を行う． 第 5 回では，2 次元 FFT を用いて画像の空間周波数を解析し，周波数領域での画像処理について理解する． 以上の実験を通して，音と画像に共通する信号処理の基本的な考え方を理解することを目的とする．

4　方法

4.1　音の工学的特徴

第 1 回では，`Matlab` を使用して正弦波を生成し，周波数，振幅，位相を変更したときの違いを確認した．

はじめに，サンプリング周波数を 16 kHz，提示時間を 1 秒として，440 Hz と 660 Hz の純音を作成した． 作成した二つの純音をそれぞれ再生した後，波形の周期の違いを確認しやすくするため，各信号の先頭 160 点，すなわち約 0.01 秒間を取り出して同じグラフ上に表示した． 横軸を時間，縦軸を振幅とし，440 Hz と 660 Hz の波形を比較した．

次に，440 Hz の純音を基準として，振幅を 0.5 倍および 0.25 倍に変更した信号を作成した． また，初期位相を 90 度および 180 度に変更した信号を作成した．位相については，90 度を $\pi/2$，180 度を π として計算に用いた． それぞれの条件について，元の 440 Hz の信号と変更後の信号の先頭 160 点を同じグラフ上に表示した． また，元の信号を左チャンネル，変更後の信号を右チャンネルとしたステレオ信号を作成し，それぞれ再生した．

うなりについては，サンプリング周波数 16 kHz，提示時間 4 秒として，440 Hz と 441 Hz の純音をそれぞれ作成した． その後，二つの信号を加算して一つの信号とした．作成した信号を再生するとともに，数秒間の時間波形を表示し，振幅が時間とともにどのように変化するかを確認した．

フーリエ級数では，周期 0.5 秒の矩形波を対象とし，基本周波数を 2 Hz とした． 基本周波数とその奇数倍の周波数を持つ正弦波を加算し，項数を N=1，5，25 と変更した． N=25 では for 文を用いて 1 次，3 次，5 次と奇数次の高調波を順番に加算した． 各条件で作成した波形を 0 秒から 2 秒まで同一グラフ上に表示し，項数を増加させたときの波形の変化を比較した．

最後に，サンプリング周波数 16 kHz，提示時間 1 秒として 16000 点のデータを用意し，randn を用いて平均 0，標準偏差 0.2 となる白色ガウス雑音を生成した． 生成した雑音を再生して 1 秒間の時間波形を表示した． また，振幅を 100 区間に分けたヒストグラムを作成し，各振幅値の出現頻度を確認した．

4.2 周波数解析

第 2 回では，Matlab の FFT を用いて時間領域の信号を周波数領域へ変換し，信号に含まれる周波数成分について確認した．

はじめに，サンプリング周波数を 8192Hz，データ数を 1024 点として 440Hz の純音を作成した．作成した信号について，0 秒から 0.02 秒までの時間波形を表示した． その後，fft 関数を用いて FFT を行い，得られた複素数データの絶対値を求めることで振幅スペクトルを作成した． 周波数軸はサンプリング周波数とデータ数から求め，0Hz から 1000Hz までの範囲を表示した．

次に，周波数を 440.2Hz に変更し，440Hz の場合と同じくサンプリング周波数 8192Hz，データ数 1024 点として純音を作成した． 作成した時間波形に FFT を行い，0Hz から 1000Hz までの振幅スペクトルを表示した． 440Hz と 440.2Hz の結果を比較することで，信号の周波数と FFT によって得られるスペクトルとの関係を確認した．

続いて，128 点の値 0 と 128 点の値 1 を一周期とし，それを 4 回繰り返すことで矩形波を作成した． このときもサンプリング周波数は 8192 Hz とした．作成した矩形波の時間波形を表示した後に FFT を行い，振幅スペクトルを求めた． 矩形波には複数の周波数成分が含まれているため，0Hz から 200Hz までの振幅スペクトルを表示し，どのような周波数にピークが現れるかを確認した．

最後に，周波数領域でのフィルタ処理を行った． サンプリング周波数を 8192Hz とし，ド，レ，ミ，ファ，ソ，ラ，シ，ドに対応する 261.38Hz，293.67Hz，329.63Hz，349.23Hz，392.00Hz，440.00Hz，493.88Hz，523.23Hz の純音をそれぞれ 0.25 秒間生成した． 作成した八つの純音を順番に連結し，一つの時間信号とした．

連結した信号を再生して時間波形を表示した後，FFT を行って周波数領域へ変換した． フィルタ処理前の振幅スペクトルは 0Hz から 600Hz まで表示した． その後，カットオフ周波数を 375Hz とするハイパスフィルタを作成した． FFT 後の正の周波数と負の周波数の両方について，絶対値が 375Hz 未満となる周波数成分を 0 とし，それ以外を 1 とした． 作成したフィルタを FFT 後の複素数データに乗算することで，低い周波数成分を除去した．

フィルタ処理後のデータに ifft を適用して時間領域へ戻し，計算結果の実部を取り出した． 最後に，フィルタ処理後の時間波形と振幅スペクトルを表示し，元の信号との違いを比較するとともに，処理後の信号を再生した．

4.3 画像の理解

第 3 回では，kut.jpg を `Matlab` で読み込み，画像の画素値に対して複数の処理を行った．

最初にカラー画像から赤，緑，青の各チャネルをそれぞれ取り出し，グレイスケール画像として表示した． その後，赤色チャネルと青色チャネルを交換したカラー画像を作成した． これによって，RGB の各成分が元画像の色にどのように関係しているかを確認した．

次にカラー画像をグレイスケール化し，量子化ビット数を 8bit，4bit，2bit，1bit に変更した．8bit 画像を基準としてビットシフトを行い，それぞれのビット数で使用できる画素値の範囲に変換した． 作成した四つの画像を並べて表示し，ビット数を減少させることで画像の見え方がどのように変化するかを確認した．

階調反転では，それぞれの量子化ビット数における最大値から各画素値を引いた． これにより，元画像の明るい部分が暗く，暗い部分が明るくなるようにした．

閾値処理では 8bit のグレイスケール画像に対して複数の閾値を設定した． 画素値が閾値より大きい場合と小さい場合に分けることで，白と黒の二値画像を作成した．

ヒストグラムでは，画素値 0 から 255 までについて，その値を持つ画素が画像内にいくつ存在するかを計算した． この課題では `hist` や `histogram` を使用せず，それぞれの画素値と画像を比較し，sum を利用して個数を求めた．

最後に，人物が写っている画像と背景だけの画像を用いて背景差分を行った． 二つの画像の同じ位置にある画素同士を減算し，差が大きく現れる場所を調べた． 差分では負の値が発生する可能性があるため，uint8 のまま単純に減算するのではなく，負の値を扱えるデータ型に変換してから計算した．

4.4 画像のフィルタ処理

第 4 回では，画像に雑音を加えた後に平滑化処理を行い，フィルタの種類による雑音除去効果の違いを確認した． また，ソーベルフィルタとラプラシアンフィルタによるエッジ検出を行い，最後に HSV 色空間を利用した肌色領域の検出を行った．

はじめに，kut.jpg を読み込み，RGB の各チャネルから

$$Y = 0.3R + 0.59G + 0.11B$$

としてグレイスケール画像を作成した．次に，この画像にインパルス雑音と白色ガウス雑音の 2 種類の雑音を加えた． インパルス雑音では，各画素に対して 0 から 1 までの一様乱数を生成し，乱数が 0.01 以下となった画素を黒，0.99 以上となった画素を白に変更した． これにより，画像中の約 1%を黒，約 1%を白とする雑音画像を作成した． 白色ガウス雑音では，平均 0，標準偏差 10 の正規乱数を生成し，元のグレイスケール画像の各画素値に加算した． 作成したグレイスケール画像，インパルス雑音画像，白色ガウス雑音画像をそれぞれ表示して比較した．

次に，作成した 2 種類の雑音画像に対して，平均化フィルタとメディアンフィルタを適用した．

平均化フィルタでは，すべての要素が 1/9 である 3×3 のフィルタを作成し，filter2 を用いて画像全体に適用した． これにより，注目画素とその周囲 8 画素を含めた 9 画素の平均値を新しい画素値として使用した． インパルス雑音画像と白色ガウス雑音画像の両方に同じ平均化フィルタを適用した．

メディアンフィルタでは，まず画像の周囲に 1 画素分のゼロパディングを行った． その後，画像を 1 画素ずつ移動しながら，それぞれの画素を中心とする 3×3 の領域を取り出した． 取り出した 9 個の画素値の中央値を median によって求め，その値を新しい画素値として代入した． この処理をインパルス雑音画像と白色ガウス雑音画像の両方に対して行い，平均化フィルタを適用した場合との違いを比較した．

続いて，グレイスケール画像にソーベルフィルタを適用してエッジを検出した． 使用した 3×3 のフィルタは，

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

および

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

の 2 種類とした． それぞれのフィルタを filter2 によってグレイスケール画像へ適用した． フィルタ処理によって得られた値には正負の値が含まれるため，絶対値を取り，それぞれの最大値を基準として 0〜255 の範囲に正規化した． その後，水平方向ソーベルフィルタと垂直方向ソーベルフィルタの結果を画像として表示して比較した．

次に，グレイスケール画像に 8 近傍ラプラシアンフィルタを適用した． 使用したフィルタは，

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

とし，filter2 を用いて画像全体に適用した． 処理後の値について絶対値を求め，最大値を基準として 0〜255 の範囲に正規化した．

さらに，ラプラシアンフィルタ適用後の画素値についてヒストグラムを作成し，画素値の分布を確認した． 正規化した画素値の下位 5%に位置する値を下限の閾値，95%に位置する値を上限の閾値として設定した． 以下の画素を 0，上限以上の画素を 255 とし，その間に存在する画素については 0〜255 の範囲へ線形変換した． これにより，弱い応答を抑えながら強いエッジを残した画像を作成し，閾値処理前と処理後の画像を比較した．

最後に，カラー画像 face.png を用いて肌色領域の検出を行った． 読み込んだ RGB 画像を rgb2hsv によって HSV 色空間へ変換し，色相 H，彩度 S，明度 V の 3 成分に分けた． 肌色として判定する範囲を，H について 0.036〜0.084，S について 0.105〜0.592，V について 0.577〜1.00 と設定した． 3 条件をすべて満たした画素を肌色領域として 1，それ以外を 0 とするマスク画像を作成した．

作成したマスクを HSV 画像の H，S，V それぞれに乗算し，肌色と判定された部分だけを残した． その後，hsv2rgb を用いて RGB 画像へ戻した． 元画像，肌色判定を表すマスク画像，肌色領域だけを残した画像をそれぞれ表示し，HSV の各成分に閾値を設定することで特定の色領域を抽出できるかを確認した．

4.5 2 次元フーリエ変換・逆フーリエ変換

第 5 回では，Matlab を用いて二次元画像に対するフーリエ変換を行い，画像の空間的な特徴と周波数成分との関係について確認した． はじめに正弦波を利用した縞模様を作成して二次元 FFT を行い，縞の本数や方向によってパワースペクトルがどのように変化するかを調べた． 次に，長方形を配置した画像についてフーリエ変換を行い，画像内で長方形の位置を変化させた場合のパワースペクトルを比較した． 最後に実際の画像に対して二次元 FFT とローパスフィルタ処理を行い，特定の周波数成分を除去した場合に画像がどのように変化するかを確認した．

はじめに，256×256 画素の画像を用意し，正弦波を利用して縞模様を作成した． 画像の列方向に沿って正弦波状に画素値を変化させることで，縦方向に伸びる縞模様を生成した． プログラムでは縞の本数を変更できるようにし，4 本，16 本，64 本の場合について確認できるようにした．今回のプログラムでは 64 本が設定されており，縞の本数に応じて表示範囲についても変更できるようにした．

作成した縞画像に fft2 を適用して二次元フーリエ変換を行い，その後 fftshift を用いて低周波成分が画像の中央に配置されるように並べ替えた． フーリエ変換後のデータからパワースペクトルを求め，値の差を確認しやすくするために対数変換して表示した． また，パワースペクトル全体だけでなく中央付近を拡大した画像も表示し，縞模様に対応する周波数成分がどの位置に現れるかを確認した．

次に，横方向に伸びる縞模様について調べた． 画像サイズは同じく 256×256 画素とし，縞の本数を 4 本，16 本，64 本の 3 種類とした． 先ほどとは異なり，画像の行方向に沿って正弦波状に画素値を変化させることで横縞を作成した． 3 種類の画像についてそれぞれ二次元フーリエ変換を行い，低周波成分を中央へ移動させた後，パワースペクトルを対数表示した． 元画像と FFT 後の結果を並べて表示することで，縞の本数を増加させた場合に周波数成分の位置がどのように変化するかを比較した．

続いて，256×256 画素の黒色画像上に白色の長方形を配置した画像を作成した． 長方形は縦方向約 30 画素，横方向約 60 画素の大きさとした． 長方形の位置については，画像中央付近に配置した場合と，そこから縦方向および横方向へ 60 画素移動させた場合の 2 種類を作成した．

それぞれの長方形画像に二次元フーリエ変換を適用し，周波数成分を画像中央へ移動させた．

その後，パワースペクトルを求めるとともに，値の違いを見やすくするために対数変換したパワースペクトルも作成した． 元の長方形画像，通常のパワースペクトル，対数変換後のパワースペクトルを横に並べ，同じ形状の長方形を画像内で移動させた場合に FFT の結果がどのように変化するかを比較した．

最後に，LENNIA.bmp を読み込み，実際の画像に対する周波数領域でのフィルタ処理を行った．読み込んだ画像に二次元フーリエ変換を適用した後，低周波成分が画像中央に配置されるように処理した．

フィルタ処理では，周波数領域の中心から一定範囲内にある成分だけを通過させる円形のフィルタを作成した． 今回のプログラムでは，中心からの距離が 50 以下となる領域を通過させ，それより外側の領域を除去するように設定した． そのため，画像の低周波成分を残し，高周波成分を除去するローパスフィルタとして処理を行った．

作成したローパスフィルタをフーリエ変換後の画像に適用し，通過させる周波数成分を制限した． その後，周波数成分の配置を元に戻して逆フーリエ変換を行い，再び画像として表示できる状態にした． 処理後のデータについて絶対値を求め，uint8 型へ変換することでフィルタ処理後の画像を作成した．

最後に，元画像，フーリエ変換後の周波数成分，対数表示したパワースペクトル，作成した円形ローパスフィルタ，フィルタ適用後の周波数成分，逆フーリエ変換によって得られた画像などをそれぞれ表示した． これらを比較することで，画像に含まれる周波数成分と，低周波成分のみを残した場合の画像の変化を確認した．

5 結果

5.1 音の工学的特徴

課題 1 では，440Hz と 660Hz の純音を作成し，0 秒から 0.01 秒までの波形を比較した． 図**??**に示すように，どちらも振幅はおよそ − 1 から 1 の範囲で変化していたが，同じ時間内では 660Hz の方が 440Hz よりも多く振動していた． 440Hz では 0.01 秒間に約 4.4 周期，660Hz では約 6.6 周期の振動が含まれるため，実際のグラフでも 660Hz の方が波の間隔が狭くなっていることが確認できた．

図 1: 440Hz と 660Hz の波形

同様に振幅を 0.25 倍にした場合には，最大振幅がおよそ 0.25 となった． この場合も波形の周期や山と谷が現れる位置は元の信号と同じであり，振幅のみが小さくなっていることが確認できた．

初期位相を 90 度変化させた場合には，変更後の波形が元の波形に対して時間方向にずれて表示された． グラフの開始時点を見ると，元の信号が 0 付近から始まっているのに対し，90 度変更した信号は最大値付近から始まっている． 一方，最大振幅や波形の周期については元の信号との違いは見られなかった．

6　機能設計１

7　機能設計２

打越です

8　はじめに

本書では, 弊社がシステム提案書で提案したスマートフォン利用支援アプリケーション「でんサポ」の詳細について示す. まず本システムの利用の流れと機能概要を説明し, その後アクティビティ図によるサブシステムの説明を行う. 次に, アプリケーション全体の画面遷移図及び, ユーザーインターフェースを示す. 最後に, データベースの設計とネットワークの構成の説明を行う.

9　運用・保守・セキュリティ対策

9.1　運用

運用の項目として以下が考えられる.

- サーバの起動・停止・再起動など，基本的な運用管理を行う.
- システムの稼働状況を常時監視し，障害発生時には運営担当者へ迅速に報告・共有する体制を整備する.
- 主な利用者が高齢者であることを踏まえ，問い合わせ窓口では平易な表現での案内を心がけ，必要に応じて問題の切り分けや報告を行う.
- 投稿された内容を定期的に確認し，誤った操作案内や不適切な投稿を発見した場合は，非表示化・削除などの対応を行う.
- タイアップ企業との連携投稿について, 掲載内容や掲載期間を管理し,「PR」である旨の表示が維持されているかを確認する.
- ユニコインやポイントの付与・消費の記録を監視し，不正な取得や残高の不整合が生じていないかを確認する.
- 定期的にデータのバックアップを実施し，障害発生時には迅速な復旧が可能な体制を整える.
- 機能追加や改善に伴うアップデートを行う際は，トラブル発生時の復元が可能となるよう，事前にバックアップデータを保存する.

9.2　保守

保守の項目として以下が考えられる.

- 運用中に発生した不具合を確認し, 原因を特定の上で修正を行う.
- システム障害が発生した場合には, 迅速に復旧対応を実施する.

- サービス継続性を高めるため, 障害時にも稼働を維持できる構成を検討する.
- AI 検索機能や AI キャラクター育成機能で利用する大規模言語モデル（LLM）の API, およびタイアップ企業等の外部サービスについて, 仕様変更や提供停止に備え, 定期的に接続状況や応答を確認する.
- 利用者の増加や投稿データの蓄積に応じて, 検索機能の応答速度が低下しないよう, 動作状況を監視し必要な改善を行う.
- セキュリティ向上および安定稼働を目的として, OS やソフトウェアを常に最新版へ更新する.
- バグ報告や機能改善要望, 不適切な投稿の通報を受け付ける窓口を設置し, 対応状況を管理する.
- ネットワークやハードウェア障害など, 専門的な対応が必要な場合は外部業者に委託する.

10　ネットワーク設計

本システムは, フロントエンドに Flutter, バックエンドに Django, データベースに PostgreSQL を用いて構築する．フロントエンドはユーザー端末上で動作し, バックエンドとの通信には HTTPS を用いる.

フロントエンドからの要求は Django が受け付け, 要求内容に応じてデータベースへのアクセスや外部 API の呼び出しを行う．会員登録やプロフィール設定, 投稿の作成・閲覧, リアクション, ユニコインやポイントの更新といった一般的な処理では, Django が PostgreSQL に対して必要なデータの参照・更新を行い, その結果をフロントエンドに返す.

AI 検索機能や AI キャラクター育成機能で LLM による処理が必要な場合は，Django が外部 LLM のクラウド API に HTTPS でリクエストを送信する．必要に応じて，API 呼び出しの前後にデータベースの参照・更新を行い，処理結果をフロントエンドに返す. この構成により, LLM の推論処理を外部サービスに委ね, バックエンド側で LLM 自体を実行・管理しない構成とする.

Django と PostgreSQL の間では，データベース接続を用いて通信する．フロントエンドから PostgreSQL および外部 LLM のクラウド API への直接接続は行わず, Django を介して処理を行う.

バックエンドおよびデータベースは，クラウド上に構築することを想定する．クラウド上の通信料は，採用するサービス，配置および通信経路などによって異なるため，構成決定時に確認する．また，外部 LLM API の利用料金についても，採用するサービスの料金体系を確認する．ユーザー端末とバックエンドとの通信では，ユーザーが契約する通信サービスの内容に応じて，ユーザー側に通信料が発生する場合がある.

11　セキュリティ対策

セキュリティ対策の項目として以下が考えられる.

- ファイアウォールや侵入検知システム（IDS/IPS）による外部からの不正アクセス防止を行う.
- Web サイトの常時 SSL 化（HTTPS 対応）により通信を暗号化する.
- 氏名やプロフィール等の個人情報を適切に管理し, パスワードはハッシュ化して保存する.

- OS・ミドルウェア・アプリケーションの脆弱性対策を定期的に実施する.
- 管理者アカウントに対して強固なパスワードポリシーや多要素認証を導入する.
- 投稿内容に個人情報や外部の不審なリンクが含まれていないかを確認し，フィッシング等の被害防止に努める.
- 利用者が詐欺被害等に遭わないよう，通報機能や注意喚起を通じて安全な利用環境の維持を図る.

12　UI 設計

aaaaaaa

13　結果

UDP トラフィックを用いて，帯域幅を 0.5Mb から 10Mb まで 0.5Mb ずつ増やしていき，それぞれのスループットとドロップ率を測定した.

以下にその結果の表およびグラフを以下に示す.

表 1: 帯域幅ごとのスループットと損失率

Bandwidth (Mb)	Throughput (Mb)	Drop Rate
0.5	0.4992	0.94
1.0	0.998	0.87
1.5	1.498	0.81
2.0	1.9972	0.75
2.5	2.4964	0.69
3.0	2.996	0.62
3.5	3.4952	0.56
4.0	3.9944	0.50
4.5	4.494	0.44
5.0	4.9932	0.37
5.5	5.4924	0.31
6.0	5.992	0.25
6.5	6.4912	0.19
7.0	6.9908	0.12
7.5	7.49	0.06
8.0	7.9892	0.00
8.5	7.9892	0.00
9.0	7.9892	0.00
9.5	7.9892	0.00
10.0	7.9892	0.00

13.1　各帯域幅とスループット

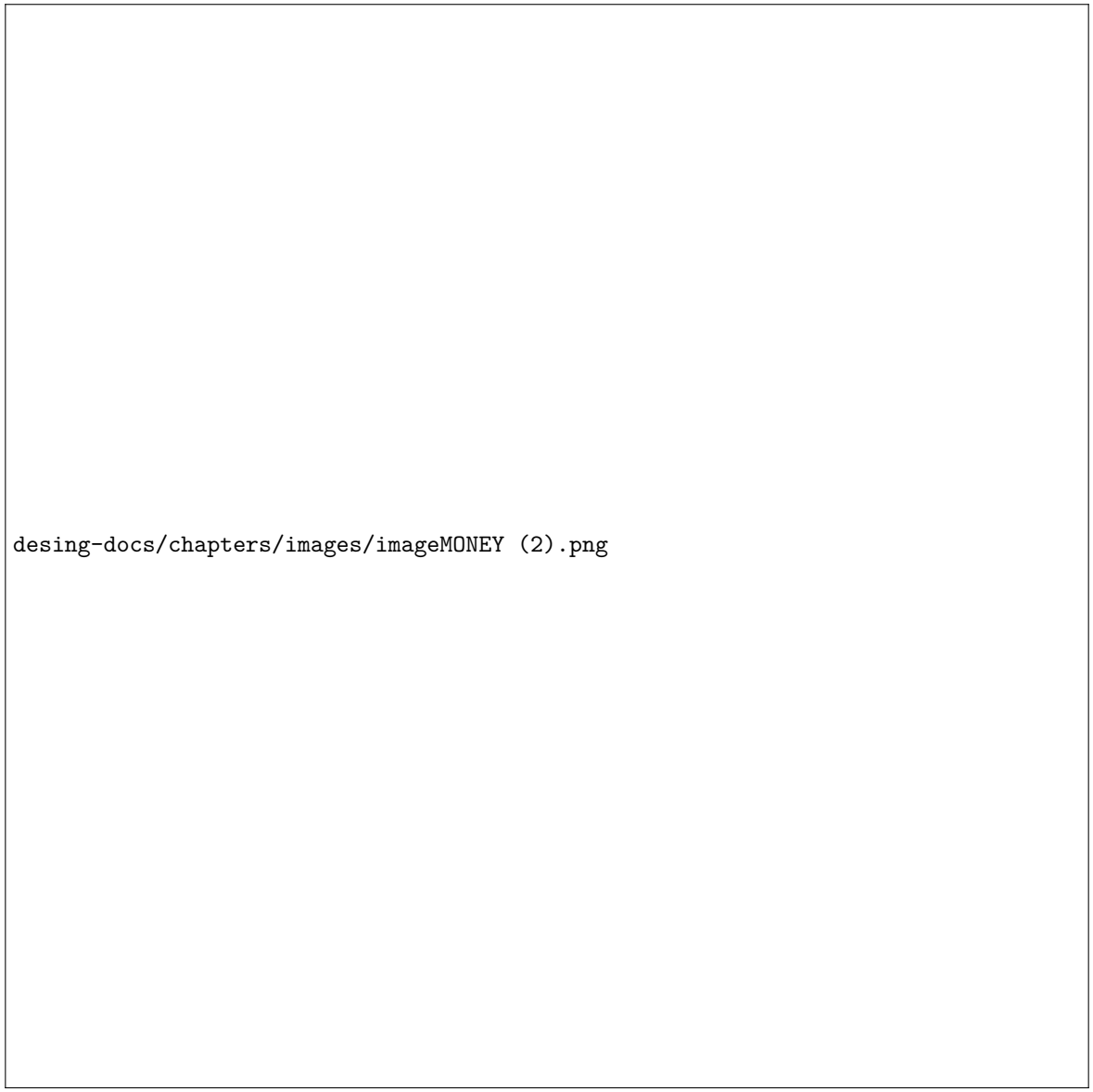


図 2: 帯域幅とスループット

aaaaaaaaaaaaa

14　利用の流れ

本システムの利用の流れを図**??**に示す．本アプリケーションは, UniCo がサービス設計およびシステム開発を行い, 運営する形で提供される．利用者には「学習者」と「投稿者」の 2 種類が存在

し,「学習者」はさらに「ゲスト」と「会員」の 2 つの利用形態に分かれる.「投稿者」および「学習者（会員)」は, 利用開始時に会員登録を行って利用を開始する. 学習者は「検索」機能でスマートフォン利用に関する不明点を検索し, 内容に合致する投稿を閲覧することができる. また, わかりやすい投稿に対しては「リアクション」機能により♥を付けることができる. さらに, 特定の投稿者に質問を行う「質問」機能により, 信頼できる投稿者から直接回答を得ることができる. 会員の学習者は, スマートフォン操作を練習する「ミニゲーム」機能をプレイする, あるいはログイン等の条件を満たすことで, アプリ内通貨である「ユニコイン」を獲得する. 獲得した「ユニコイン」は, 「キャラクター交流・育成」機能におけるキャラクターの着せ替え等に利用することができる. 投稿者は「投稿」機能で, スマートフォン利用に関する知識をテキストと画像を用いて公開する. 「投稿評価確認」機能では, 投稿を行ったときと, 学習者のリアクション機能により得られた♥とに基づいて「Pt（ポイント)」を獲得し, Pt の合計を他の投稿者と比較することができる’ ランキング’ 形式での表示や, ’ 感謝の木’ として, 学習者から得た♥の数に応じて成長する木を表示し, 投稿者のモチベーション維持を図る機能を含む. また,’ 投稿者ランク’ の確認を行うことが出来る. また, 本システムでは運営である UniCo が企業とタイアップを行い, 当該企業のアプリケーションやサービスの利用方法に関する投稿を作成し, 企業名義のユーザーとして公開する「広告・タイアップ管理投稿」機能を備える. これにより, 学習者は信頼性の高い情報を得られるとともに, 運営はタイアップによる収益を得ることができる. さらに, 本システムの利用を通じて高齢者のスマートフォン利用能力の向上と心理的抵抗感の解消を促し, 誰もがデジタル化された社会の情報やサービスへ平等にアクセスできる環境の実現につなげる.

システム提案書

でんサポ

第 1.0 版

2026 年 11 月 13 日

UniCo

目次

参考文献

1. 内閣府本府．令和 8 年版高齢社会白書（全体版）．閲覧日：令和 8 年 10/15
2. 総務省，令和 8 年度版 情報通信白書（年次報告書）｜インターネット．閲覧日：令和 8 年 10/15
3. 総務省，令和 3 年度版 情報通信白書（年次報告書）｜デジタル活動支援．閲覧日：令和 8 年 10/15
4. 総務省，令和 8 年度版 情報通信白書（年次報告書）｜情報通信機器・端末．閲覧日：令和 8 年 10/15